`...` that marks the pack expansion at the end of the pattern. For example, rather than just creating a tuple of the elements supplied in your parameter pack, you can create a tuple of pointers to the elements or even a tuple of `std::unique_ptr<>`s to your elements:

```
template<typename ... Params>
struct dummy3
{
    std::tuple<Params* ...> pointers;
    std::tuple<std::unique_ptr<Params> ...> unique_pointers;
};
```

The type expression can be as complex as you like, provided the parameter pack occurs in the type expression, and provided the expression is followed by the `...` that marks the expansion. When the parameter pack is expanded, for each entry in the pack that type is substituted into the type expression to generate the corresponding entry in the resulting list. Thus, if your parameter pack `Params` contains the types `int,int,char`, then the expansion of `std::tuple<std::pair<std::unique_ptr<Params>,double> ... >` is `std::tuple<std::pair<std::unique_ptr<int>,double>,std::pair<std::unique_ptr<int>,double>, std::pair<std::unique_ptr<char>,double> >`. If the pack expansion is used as a template argument list, that template doesn't have to have variadic parameters, but if it doesn't, the size of the pack must exactly match the number of template parameters required:

```
template<typename ... Types>
struct dummy4
{
    std::pair<Types...> data;
};
dummy4<int,char> a;
dummy4<int> b;
dummy4<int,int,int> c;
```

❶ OK, data is std::pair<int,char>

❷ Error, no second type

❸ Error, too many types

The second thing you can do with a pack expansion is use it to declare a list of function parameters:

```
template<typename ... Args>
void foo(Args ... args);
```